

Rapport Technique — Plateforme de Gestion d'Événements LinSoft

Projet de Fin d'Études (PFE) — Maryem Achouri Encadrement : LinSoft Date du rapport : Juin 2026

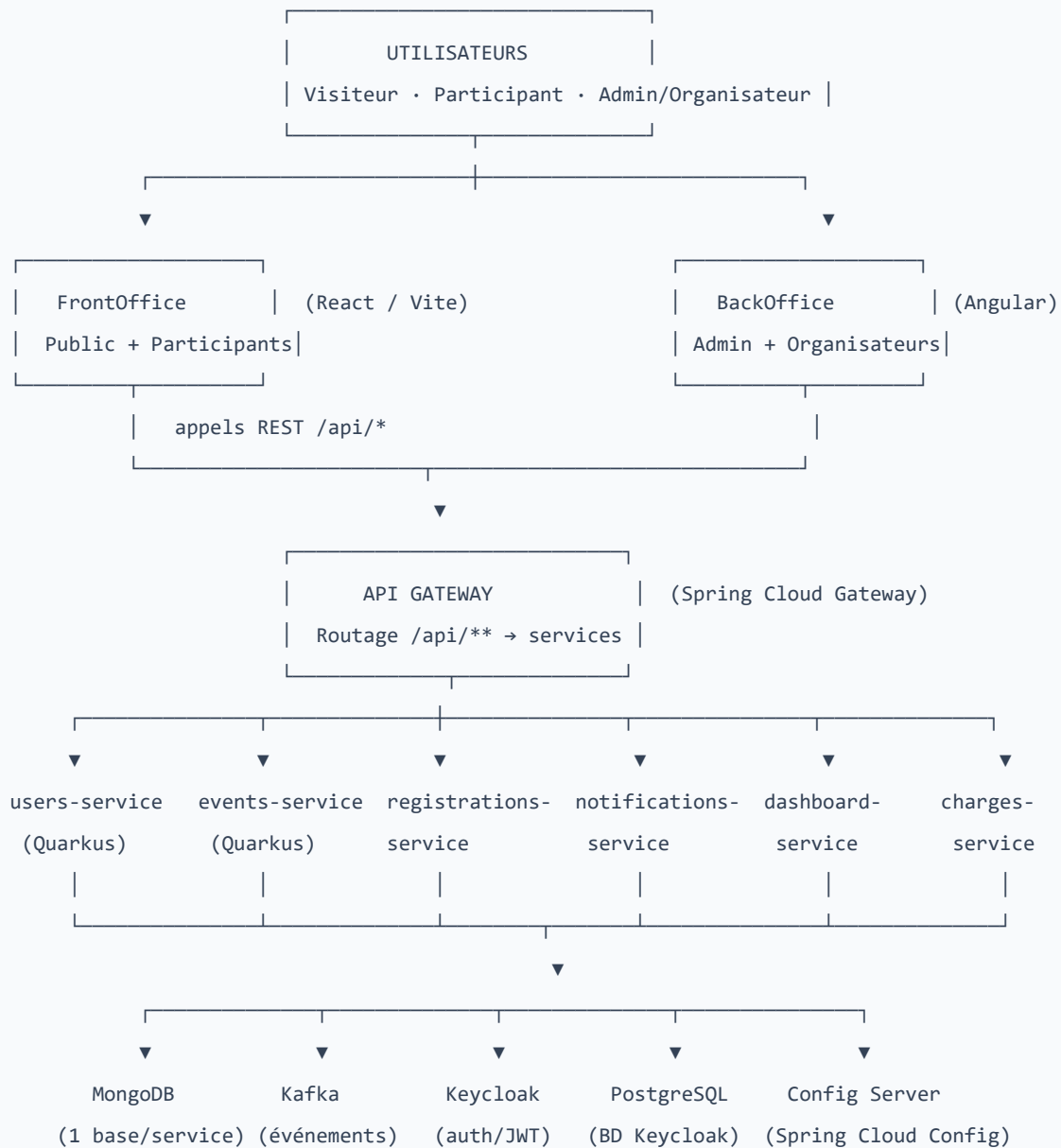
1. Présentation générale

La plateforme **Event Management** est une application web de **gestion d'événements** développée pour LinSoft. Elle permet :

- aux **visiteurs** de parcourir les événements publics ;
- aux **participants** de s'inscrire à des événements (avec ou sans compte) ;
- aux **organiseurs** et **administrateurs** de créer, gérer et suivre les événements, les inscriptions et les charges associées.

Le système est bâti sur une **architecture microservices** (backend Java) avec **deux interfaces** distinctes (FrontOffice public et BackOffice d'administration), une **authentification centralisée** (Keycloak), une **communication événementielle** (Kafka), et un **déploiement cloud** sur **OpenShift**.

2. Architecture globale



Patterns architecturaux mis en œuvre : - **Microservices** (services autonomes, déployables indépendamment) ; - **API Gateway** (point d'entrée unique, routage, CORS) ; - **Database per Service** (chaque service possède sa propre base MongoDB) ; - **Event-Driven** (communication asynchrone via Kafka) ; - **Externalized Configuration** (Spring Cloud Config) ; - **Identity Provider centralisé** (Keycloak / OIDC).

3. Stack technologique

Couche	Technologies
Langage backend	Java 21
Frameworks backend	Quarkus 3.8.4 (services métier), Spring Boot 3.2.5 + Spring Cloud 2023.0.1 (infra)
FrontOffice	React 18.2 · Vite 5.2 · TypeScript 5.2 · React Router 6 · Axios · Leaflet
BackOffice	Angular 12.1 · Black Dashboard · keycloak-angular 9.3 / keycloak-js 18 · ngx-toastr · Firebase · Leaflet
Authentification	Keycloak 24 (OpenID Connect / OAuth2, JWT)
Bases de données	MongoDB 7 (données métier) · PostgreSQL 16 (Keycloak)
Messaging	Apache Kafka (mode KRaft sur OpenShift)
Cartographie	Leaflet + OpenStreetMap / Nominatim (sans clé API)
Notifications	Email (SMTP Gmail) · SMS (Africa's Talking) · Push (Firebase Cloud Messaging)
Conteneurisation	Docker (local) · OpenShift / Kubernetes (cloud)
CI de build	OpenShift BuildConfig (build des images sur le cluster depuis Git)

4. Backend — Microservices

4.1 Services d'infrastructure (Spring Boot / Spring Cloud)

Service	Port	Rôle
config-server	8888	Configuration centralisée (Spring Cloud Config, backend <i>native</i> embarqué)
gateway-service	8080	Point d'entrée unique, routage <code>/api/**</code> , CORS, agrégation
eureka-server	8761	Service discovery (utilisé en local ; remplacé par le DNS Kubernetes sur OpenShift)

4.2 Services métier (Quarkus)

Service	Port	Base MongoDB	Rôle
users-service	8083	users_db	Gestion des utilisateurs, authentification (login/refresh), intégration Keycloak (admin API + OIDC)
events-service	8081	events_db	CRUD des événements, géolocalisation, publication d'événements Kafka
registrations-service	8082	registrations_db	Inscriptions (participants et visiteurs invités), gestion des places
notifications-service	8084	notifications_db	Envoi d' emails , SMS et notifications push (déclenché par Kafka)
dashboard-service	8085	dashboard_db	Agrégation de statistiques et tableaux de bord
charges-service	8086	charges_db	Gestion des charges/coûts d'événements + prédiction IA des charges

Technologies Quarkus utilisées : `resteasy-reactive`, `mongodb-panache`, `oidc` (sécurité JWT), `smallrye-reactive-messaging-kafka`, `mailer`.

5. Communication inter-services

- **Synchrone (REST)** : le FrontOffice/BackOffice appellent la **gateway** (`/api/...`), qui route vers le service concerné (par DNS Kubernetes en production : `http://users-service:8083`, etc.).
- **Asynchrone (Kafka)** : les services publient/consomme des événements métier. Exemple : la création d'un événement (`event.created`) est consommée par `notifications-service` (qui met en cache le titre de l'événement pour les emails) ; une inscription déclenche l'envoi d'une notification.
- **Database per Service** : chaque microservice possède sa propre base MongoDB, garantissant le découplage des données.

6. Authentification & Sécurité

6.1 Keycloak (Identity Provider)

- **Realm** : `event-mgmt`
- **Client** : `users-service` (confidentiel, *Direct Access Grants* activé pour le flux ROPC)
- **Rôles** : `admin`, `participant`, `event-organizer` (+ variantes `organisateur` / `organizer`), `user`
- **Base de données** : PostgreSQL

6.2 Login unifié (conception)

Le projet implémente un **login unifié côté FrontOffice**, sans formulaire Keycloak visible :

1. L'utilisateur saisit ses identifiants dans le **FrontOffice**.
2. Le FrontOffice appelle `/api/auth/login` → `users-service` exécute un **flux ROPC** (Resource Owner Password Credentials) contre Keycloak en interne et renvoie les tokens (JWT).
3. **Redirection par rôle** : - rôle **participant** → reste dans le **FrontOffice** ; - rôle **admin / organisateur** → redirigé vers le **BackOffice**, avec passage des tokens (jamais de formulaire Keycloak).
4. Le **BackOffice** injecte le token reçu dans keycloak-js et rafraîchit via `users-service` (le token étant émis pour le client `users-service`).

6.3 Validation des tokens

Les services protégés (Quarkus OIDC, `application-type=service`) valident les **JWT** : signature, expiration et **émetteur** (*issuer*). Sur OpenShift, l'émetteur est **interne et cohérent** (`http://keycloak:8080/realms/event-mgmt`), tout l'échange Keycloak se faisant en interne au cluster.

7. Frontends

7.1 FrontOffice (React / Vite) — port 4300

Public et participants : - Page d'accueil et **catalogue d'événements** ; - **Détail d'un événement** avec **carte interactive** (Leaflet/OpenStreetMap) localisant le lieu ; - **Inscription participant et visiteur invité** (l'invité conserve son inscription via `localStorage` pour pouvoir l'annuler) ; - **Login unifié** + tableau de bord « Mes inscriptions » + profil ; - Notifications visuelles (toasts), navbar responsive.

7.2 BackOffice (Angular / Black Dashboard) — port 4200

Administrateurs et organisateurs : - **Gestion des événements** : création/édition avec **sélection du lieu sur une carte** (recherche d'adresse + clic pour positionner) ; - **Gestion des inscriptions** : participants regroupés par événement, notification « places complètes », annulation/suppression ; - **Gestion des utilisateurs, gestion des charges + prédiction IA, tableau de bord et dashboard IA** ; - Interface entièrement **traduite en français**.

8. Fonctionnalités clés

- **Événements** : CRUD complet, catégories, statut (brouillon/publié), événements en ligne (lien Google Meet privé), photo, **géolocalisation cartographique**, capacité maximale.
- **Inscriptions** : participant authentifié **ou** visiteur invité ; détection « événement complet » ; annulation.

- **Notifications multicanal** : **email** (confirmation d'inscription, avec le **nom réel** de l'événement), **SMS** (Africa's Talking), **push** (Firebase).
 - **Charges & IA** : suivi des coûts d'événements et **prédiction** des charges (modèle de machine learning).
 - **Tableaux de bord** : statistiques agrégées.
 - **Cartographie** : Leaflet + OpenStreetMap/Nominatim — **aucune clé API requise**.
-

9. Déploiement

9.1 Local — Docker Compose

L'ensemble (services + Keycloak + PostgreSQL + MongoDB + Kafka + Zookeeper + MailHog + Kafka UI) se lance via `docker-compose.yml`. Les images des services Java sont construites par des **Dockerfiles multi-étapes** (compilation Maven dans le conteneur, exécution sur une image JRE Alpine).

9.2 Cloud — OpenShift

Le backend complet a été **déployé et validé sur un cluster OpenShift** (accès via VPN). Points techniques :

- **Build sur le cluster (BuildConfig)** : OpenShift **clone le dépôt Git** et construit chaque image (stratégie Docker, multi-étapes Maven) qu'il **stocke dans le registre interne** — aucune image locale n'est poussée.
 - **Sécurité OpenShift (SCC restricted)** : tous les conteneurs tournent en **non-root** (UID assigné par le cluster). Choix d'images compatibles : MongoDB officiel (UID arbitraire), **Kafka en mode KRaft** (image Bitnami, sans Zookeeper), Keycloak 24 (non-root), PostgreSQL Red Hat (image OpenShift native).
 - **Service Discovery natif** : sur OpenShift, **Eureka est supprimé** au profit du **DNS Kubernetes** (chaque `Service` est résolvable par son nom). La gateway route en `http://service:port` au lieu de `lb://`.
 - **Exposition** : les composants publics (gateway, Keycloak, frontends) sont exposés via des **Routes** OpenShift (TLS edge) ; les services internes communiquent via leurs `Service` ClusterIP.
 - **Gestion des ressources** : le projet est soumis à un **quota** (4 cœurs CPU, 10 Gi RAM). Limites CPU/mémoire ajustées par composant + `startupProbe` pour absorber les démarrages lents des JVM bridées en CPU.
 - **Stockage persistant** : volumes **PVC** (Ceph RBD) pour MongoDB, Kafka et PostgreSQL.
 - **Infrastructure as Code** : tous les déploiements sont décrits en **manifestes YAML** versionnés (dossier `openshift/`).
-

10. Structure du projet

```
Event Management/
├─ services/                # Backend microservices (Maven multi-module)
│   ├─ eureka-server/      # Service discovery (local)
│   ├─ config-server/      # Configuration centralisée
│   ├─ gateway-service/    # API Gateway
│   ├─ users-service/      # Utilisateurs + authentification
│   ├─ events-service/     # Événements
│   ├─ registrations-service/ # Inscriptions
│   ├─ notifications-service/ # Notifications (email/SMS/push)
│   ├─ dashboard-service/  # Tableaux de bord
│   └─ charges-service/    # Charges + prédiction IA
├─ FrontOffice/            # Application React (public/participants)
├─ BackOffice/back-office/ # Application Angular (admin/organisateurs)
├─ openshift/              # Manifestes de déploiement OpenShift (YAML)
├─ docker-compose.yml      # Orchestration locale
└─ keycloak-theme/         # Thème Keycloak personnalisé
```

11. Défis techniques résolus

Défi	Solution apportée
Login unifié sans formulaire Keycloak	Flux ROPC côté serveur (<code>users-service</code>), redirection par rôle avec passage de token, injection dans keycloak-js côté BackOffice
Email affichant l'ID au lieu du nom	Cache du titre d'événement dans <code>notifications-service</code> via consommation Kafka (<code>event.created</code>)
Cluster OpenShift sans push d'images	Build sur le cluster depuis Git (BuildConfig) → registre interne
Sécurité non-root (SCC restricted)	Images compatibles UID arbitraire ; Kafka KRaft (Bitnami) ; PostgreSQL Red Hat ; pas de <code>runAsUser</code> fixe
Images non tirables à l'exécution	Noms d'images pleinement qualifiés (<code>docker.io/...</code>) pour contourner la redirection du registre interne
Quota CPU serré (4 cœurs)	Limites CPU basses + <code>startupProbe</code> pour ne pas tuer les JVM au démarrage
Découverte de services	Suppression d'Eureka, routage par DNS Kubernetes (cloud-native)
Cohérence de l'émetteur OIDC	Issuer interne unique (<code>keycloak:8080</code>) pour tout l'échange serveur-à-serveur

12. État d'avancement

-  Backend complet (9 services) **déployé et fonctionnel sur OpenShift** ;
 -  Infrastructure (Keycloak, PostgreSQL, MongoDB, Kafka) **opérationnelle sur le cluster** ;
 -  Realm Keycloak `event-mgmt` recréé (clients, rôles, utilisateur admin) ;
 -  Gateway routant correctement via le DNS Kubernetes (`/api/events` validé) ;
 -  **Frontends** (FrontOffice React + BackOffice Angular) : conteneurisation et déploiement OpenShift **en cours**.
-

Document généré dans le cadre du PFE — plateforme Event Management (LinSoft).